

General Information

- Class time (NC 244)

Wed (12.00-13.00), THU (11.00-12.00), Friday (9.00-11.00)

- Lab (Friday, 2.00pm-5.00pm)

Software Lab

General Information

- **Grading Policy**

- Midsem – 30%
- Endsem – 30%
- Internal – 30-40% (Tutorial, Class tests, and Attendance)

- <http://cse.iitkgp.ac.in/~bivasm/OS2026.html>

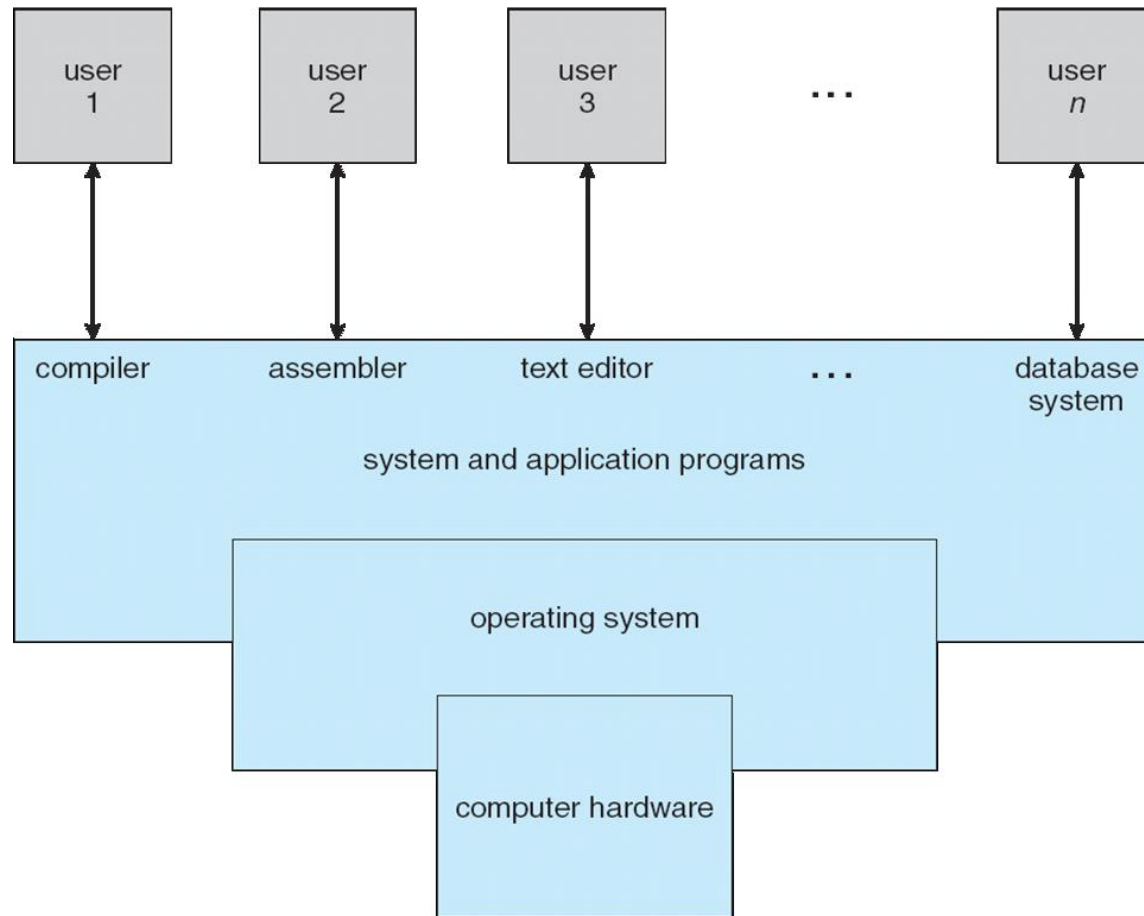
- **Text Books / References :**

1. Silbersehatz A. and Galvin P., “Operating System Concepts”, Wiley.
2. Tanenbaum A.S., “Operating System Design & Implementation”, Practice Hall NJ.
3. Stalling, William, “Operating Systems”, Maxwell McMillan International Editions, 1992.
4. Dietel H. N., “An Introduction to Operating Systems”, Addison Wesley.

Role of Operating system

- Computer system can be divided into four components:
 - Hardware – provides basic computing resources
 - CPU, memory, I/O devices
 - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
 - Word processors, compilers, web browsers, database systems, video games
 - Users
 - People, machines, other computers
 - Operating system
 - Controls and coordinates use of hardware among various applications and users

Four Components of a Computer System



What is an Operating System?

(User's view)

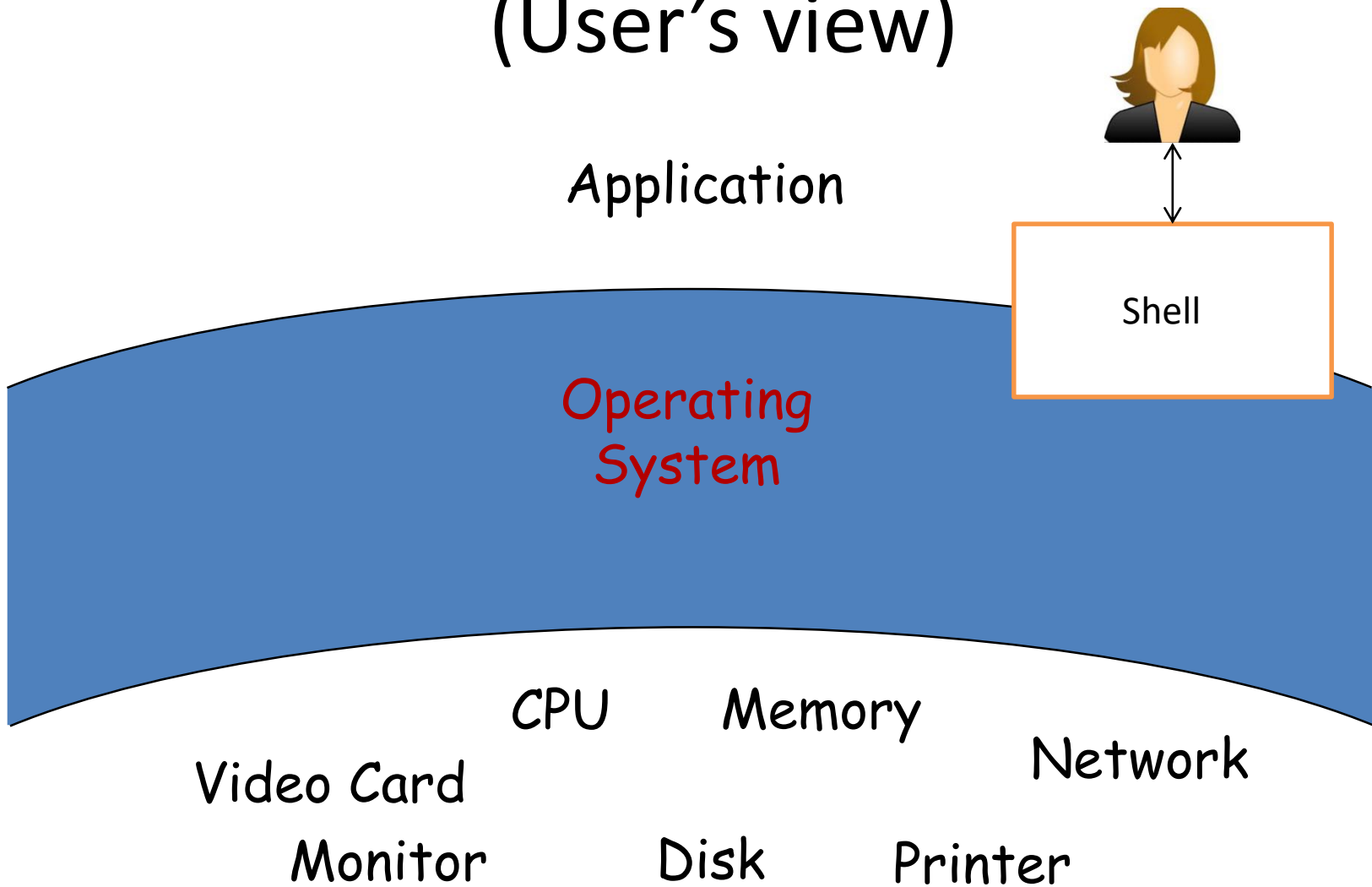
- A program that acts as an intermediary between a **user** of a computer and the **computer hardware**
- Defines an **interface** for the user to use services provided by the system
- Creates an environment for the user

What is an Operating System?

(User's view)

- Abstract Machine
 - Hides complex details of the underlying hardware
 - Provides common API to applications and services
 - Simplifies application writing
- Command Interpreter
 - Part of a OS that understands and executes commands that are entered interactively by a human being or from a program
 - Shell

What is an Operating System? (User's view)



Why is abstraction important?

- Without OSs and abstract interfaces, application writers must program all device access directly
 - load device command codes into device registers
 - understand physical characteristics of the devices
- Applications suffer!
 - very complicated maintenance and upgrading
 - no portability

What Operating Systems Do (User's view)

Depends on the point of view

- Single user system
- Users want convenience, **ease of use**
 - Don't care about **resource utilization**

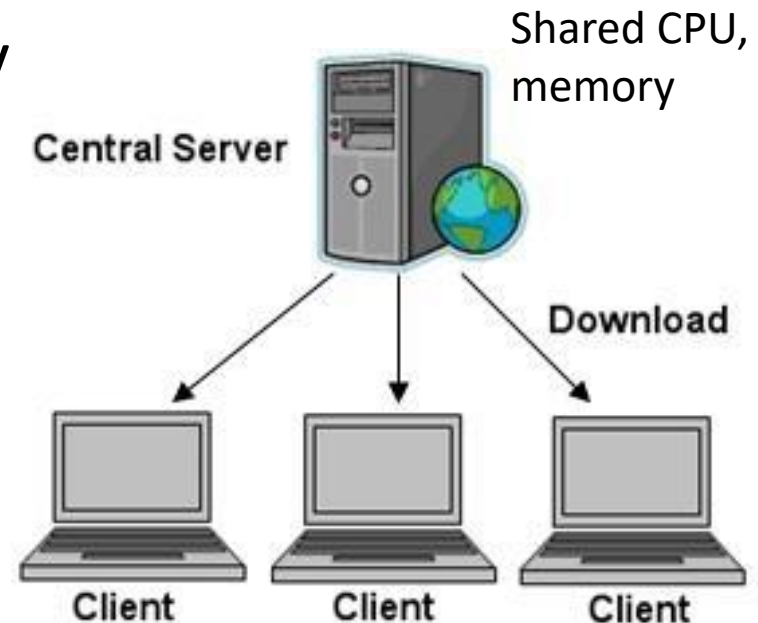


Optimized for single user
experience

What Operating Systems Do (User's view)

Depends on the point of view

- Shared computer such as **mainframe** must keep all users happy
- Response time minimum
 - Keep all the users happy



What Operating Systems Do (Systems view)

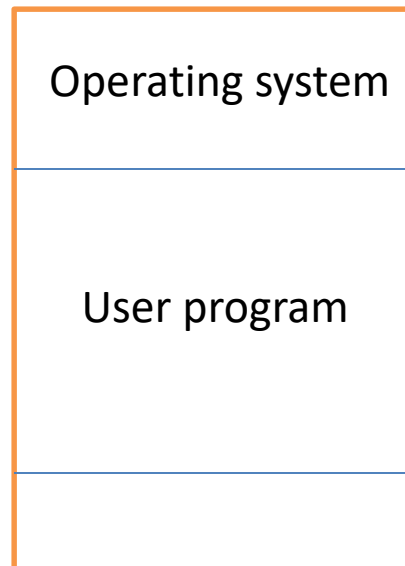
- OS is a **resource allocator**
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
 - Controls execution of programs to prevent errors and improper use of the computer

Concept of Process

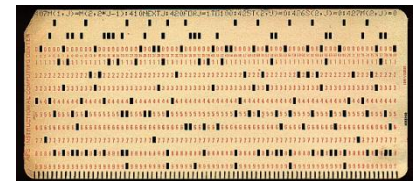
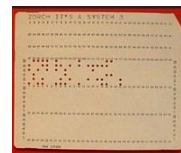
- Process
 - Program loaded in memory and in execution
- Program is a passive entity
- Process is an active entity

Types of Systems

- Batch Systems
 - Multiple jobs, but only one job in memory at one time and executed (till completion) before the next one starts

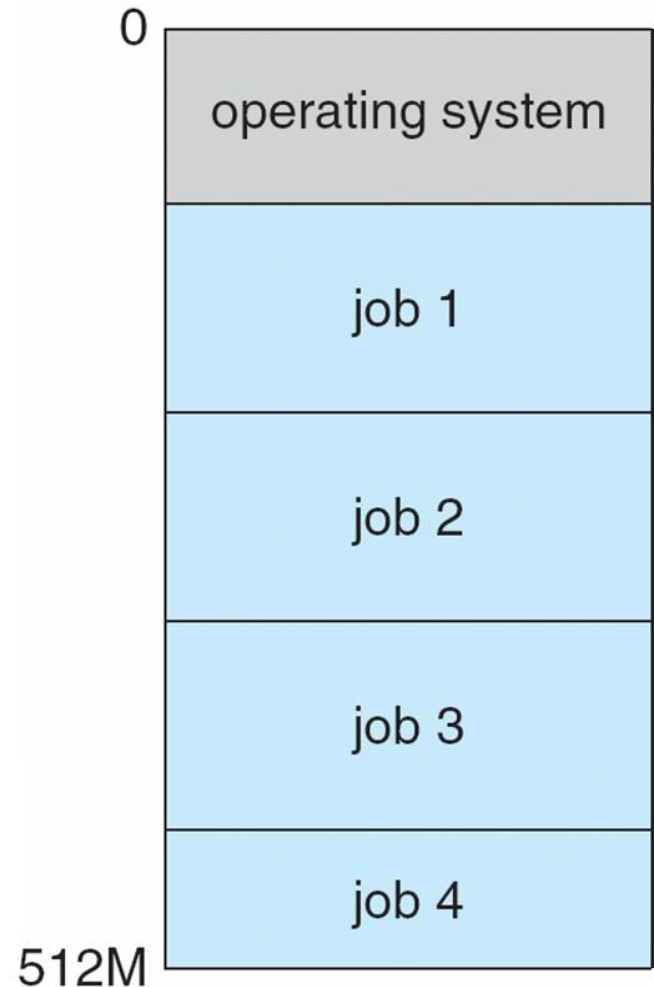


Jobs waiting



Types of Systems

- Multiprogrammed Systems
 - Multiple jobs in memory, CPU is multiplexed between them
 - Single user cannot keep CPU and I/O devices busy at all times
 - When it has to wait (for I/O for example), OS switches to another job
 - Multiprogramming ensures -- CPU always has one job to execute
 - A subset of total jobs in system is kept in memory
 - One job selected and run via **job scheduling**
- **Effective resource utilization**
- **Poor user experience**

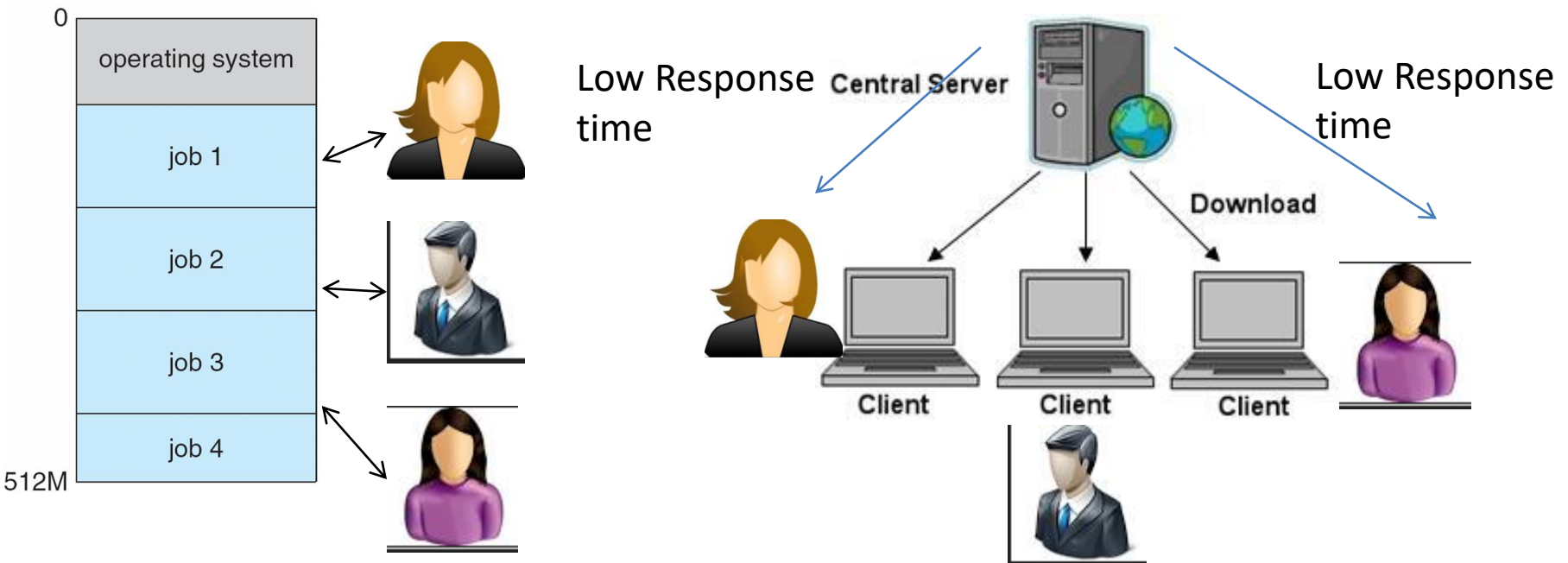


Types of Systems

- Time-sharing Systems (**multitasking**)

logical extension of multiprogramming in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing

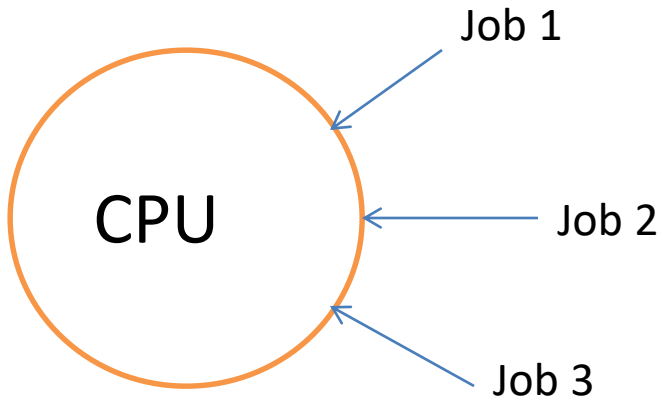
- **Response time** should be < 1 second
- Each user has at least one program executing in memory
- If several jobs ready to run at the same time $\Rightarrow$ **CPU scheduling**



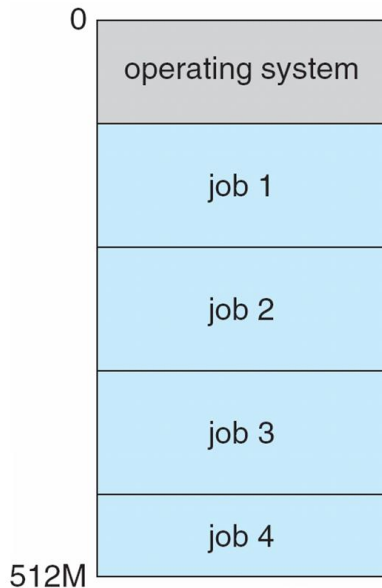
What Operating Systems Do (Systems view)

- OS is a **resource allocator**
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
 - Controls execution of programs to prevent errors and improper use of the computer

Manage resources



1. Share the CPU with several users
2. Decide when to allocate CPU to which user (CPU scheduling)
3. Ensure fair user experience



Memory

1. Share memory with several different users
2. Should not overlap
3. Ensure protection

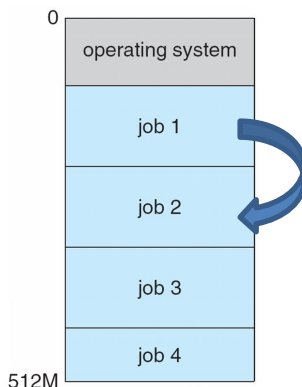
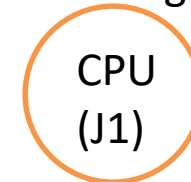
What Operating Systems Do (Systems view)

- OS is a **resource allocator**
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
 - Controls execution of programs to prevent errors and improper use of the computer

Control program (Protection)

- Multiple jobs are sharing the common resource
 - With sharing, many processes could be adversely affected by a bug in one program
 - Make sure that error in one program could cause problems only for that program
 - A job gets stuck in an infinite loop
 - Prevent correct operations of other jobs
- One erroneous program might modify another program, even operating system

J2, J3
waiting



Incorrect program cannot
cause other programs to
execute incorrectly

Execution of OS

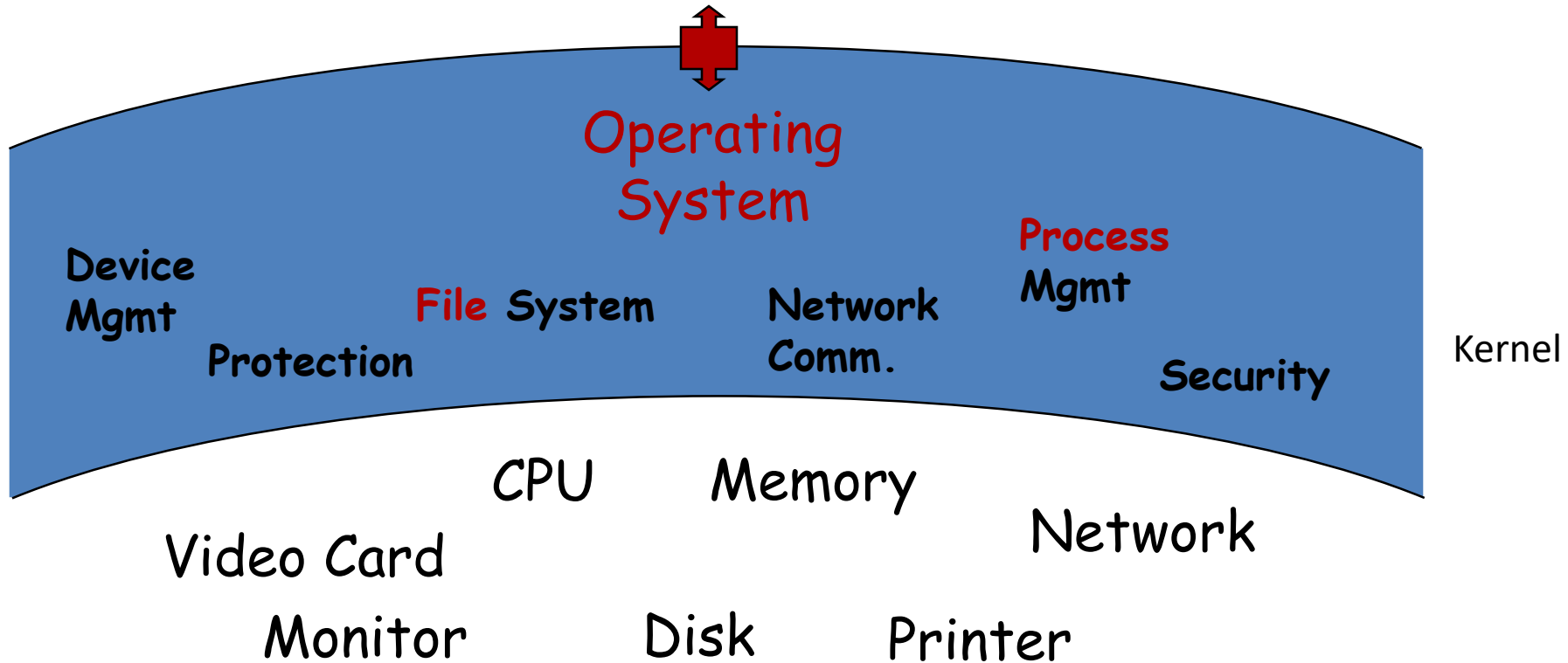
- Interrupt driven
- Until an interrupt comes, OS remains Idle
- Interrupt/trap
 - Possibility 1---- error
 - Possibility 2
 - User program invokes OS code by generating Interrupt, system call
 - To perform some task reserved for OS
 - Accessing I/O devices (read, write files)

ISR

Providing abstraction via system calls

Application

System Calls: fork(), wait(), read(), open(), write(), mkdir(), kill() ...



Execution of OS

- Interrupt driven
- Until an interrupt comes, OS remains Idle
- User program invokes OS code by generating Interrupt, system call
 - To perform some task reserved for OS
 - Accessing I/O devices (read, write files)
- Any difference in execution between user and OS program?

Operating-System Operations

- Must distinguish between the use level code and OS code
 - **User mode** and **kernel mode**
 - **Mode bit** provided by hardware
 - Provides ability to distinguish when system is running user code or kernel code
 - System call changes mode to kernel, return from call resets it to user

System boot



Hardware starts kernel mode



Load Operating system

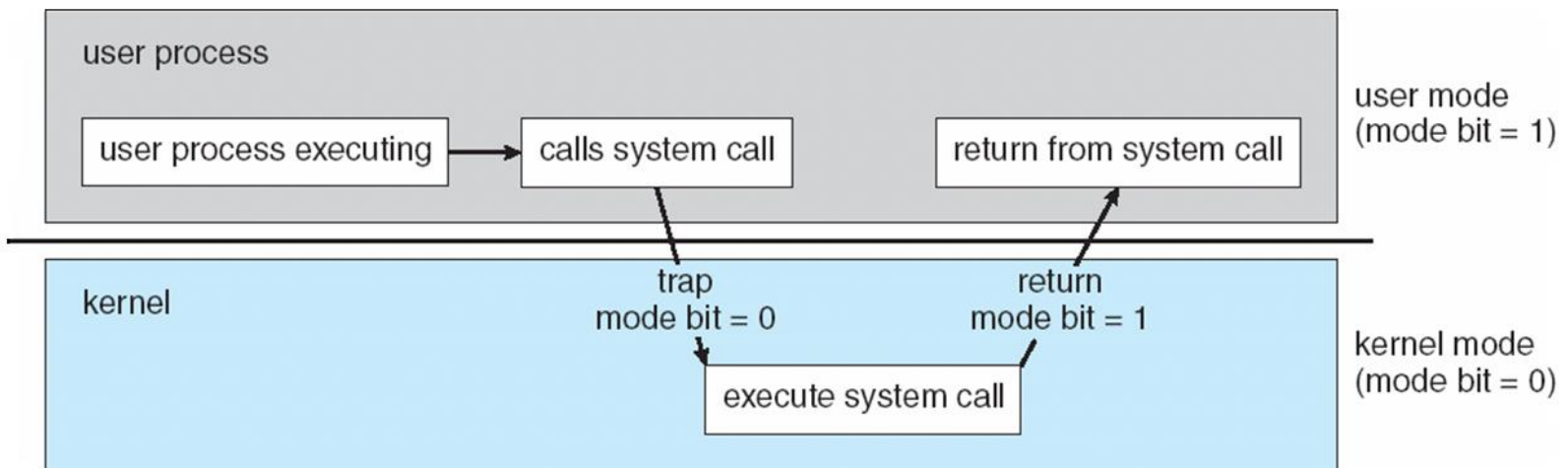


Start user application



Switch to User mode

Whenever Trap or interrupt occurs, hardware switches to user to kernel mode



Some instructions designated as **privileged**, only executable in kernel mode

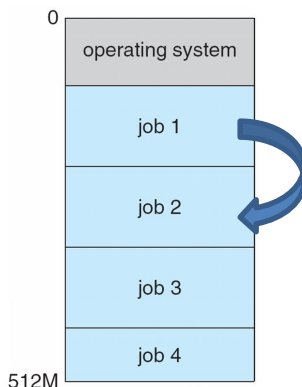
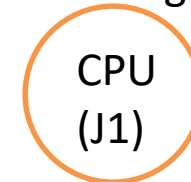
What Operating Systems Do (Systems view)

- OS is a **resource allocator**
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
 - Controls execution of programs to prevent errors and improper use of the computer

Control program (Protection)

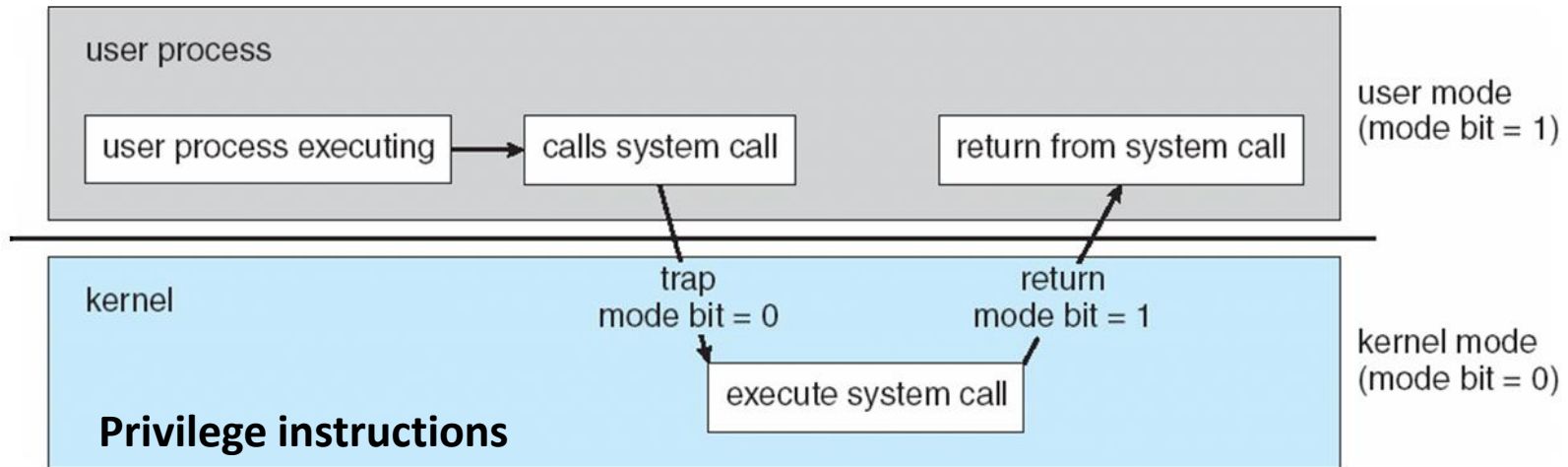
- Multiple jobs are sharing the common resource
 - With sharing, many processes could be adversely affected by a bug in one program
 - Make sure that error in one program could cause problems only for that program
 - A job gets stuck in an infinite loop
 - Prevent correct operations of other jobs
- One erroneous program might modify another program, even operating system

J2, J3
waiting



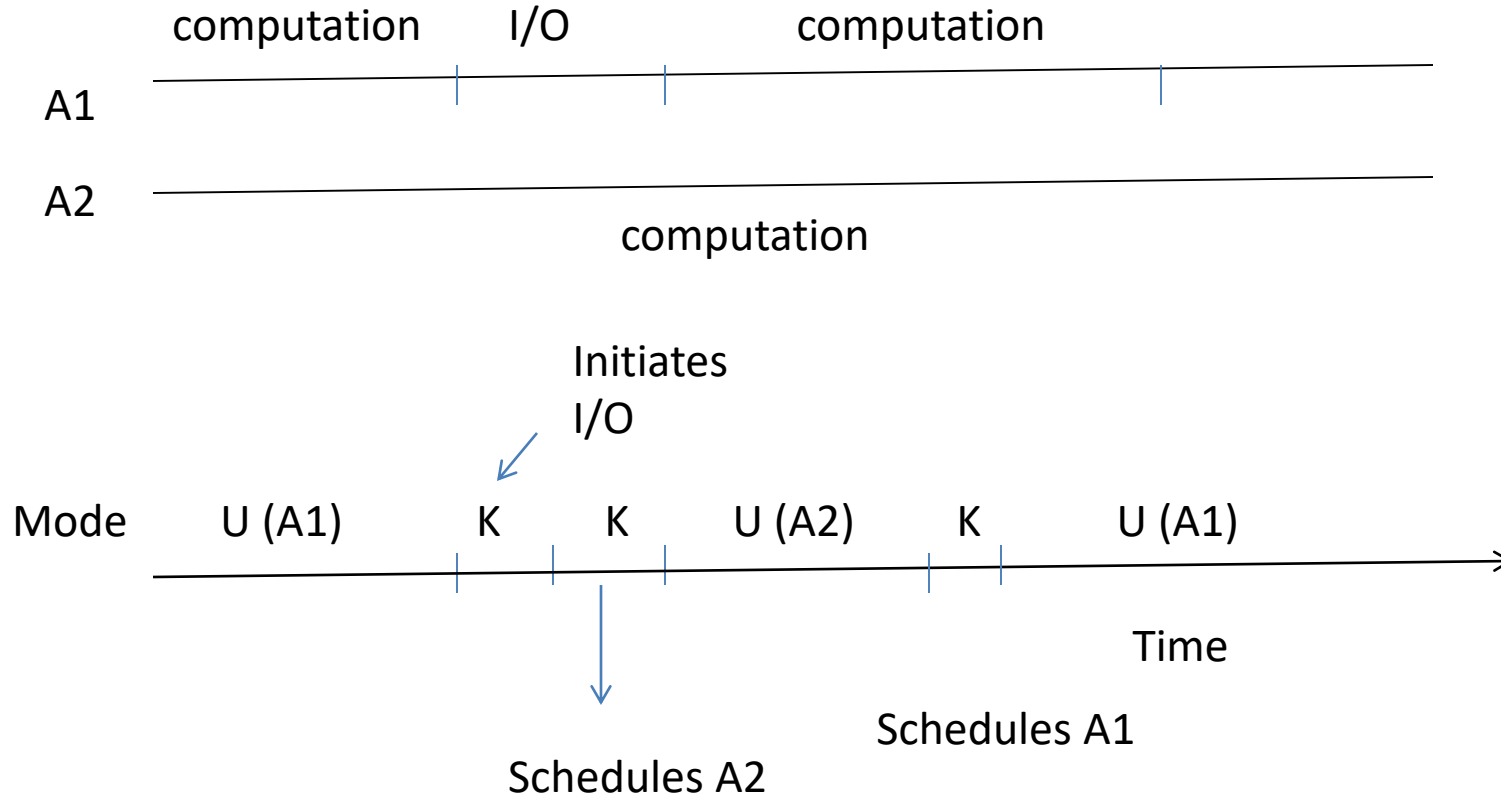
Incorrect program cannot
cause other programs to
execute incorrectly

Dual-mode operation allows OS to protect itself and other system components



- Software error creates **exception** or **trap**
 - Division by zero, request for operating system service, setting timer
- Restricts user process from executing **privilege instruction**
- E.g. Segmentation fault!

Mode change



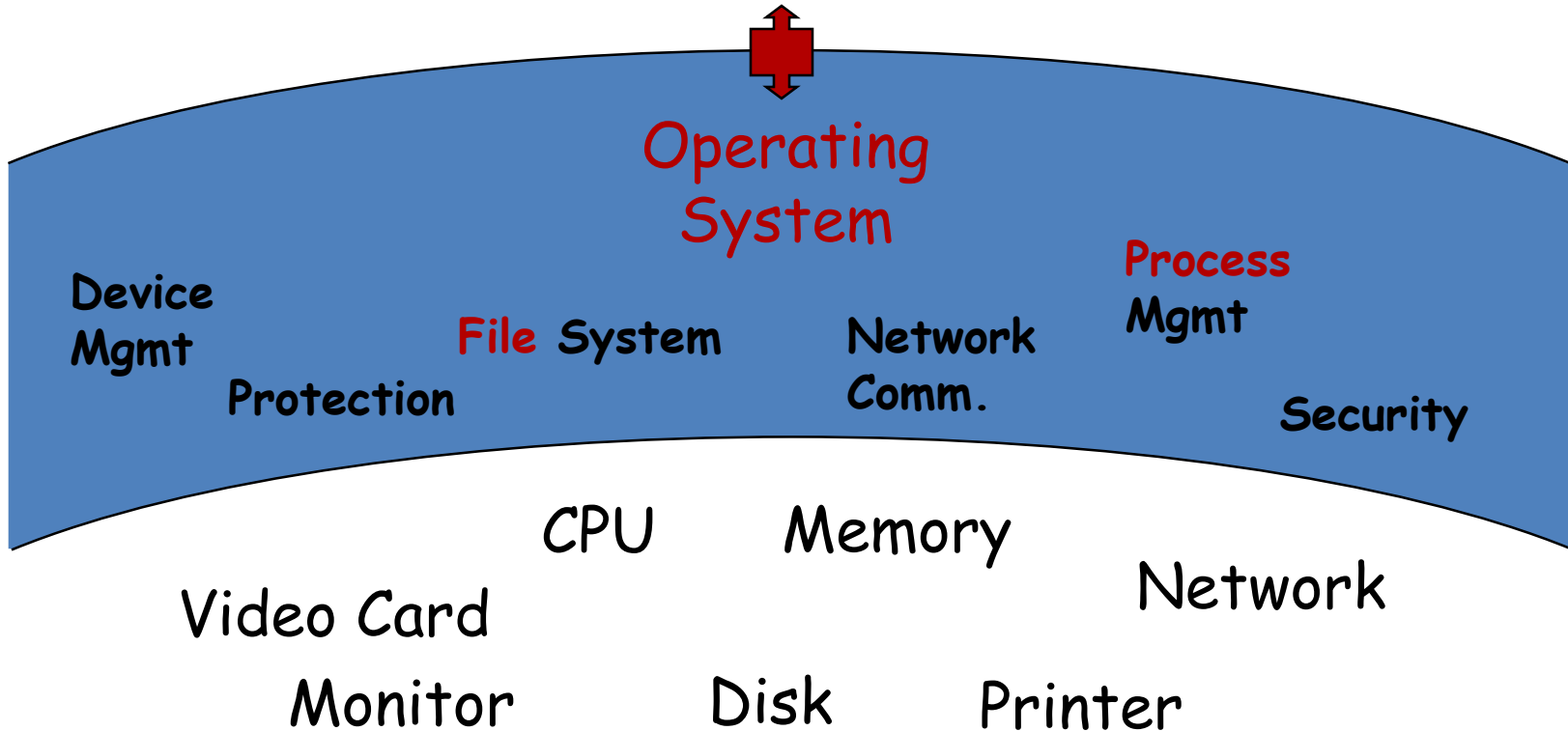
What is the difference between
Privileged Instruction and System call?

System calls

Win 32
POSIX
JVM

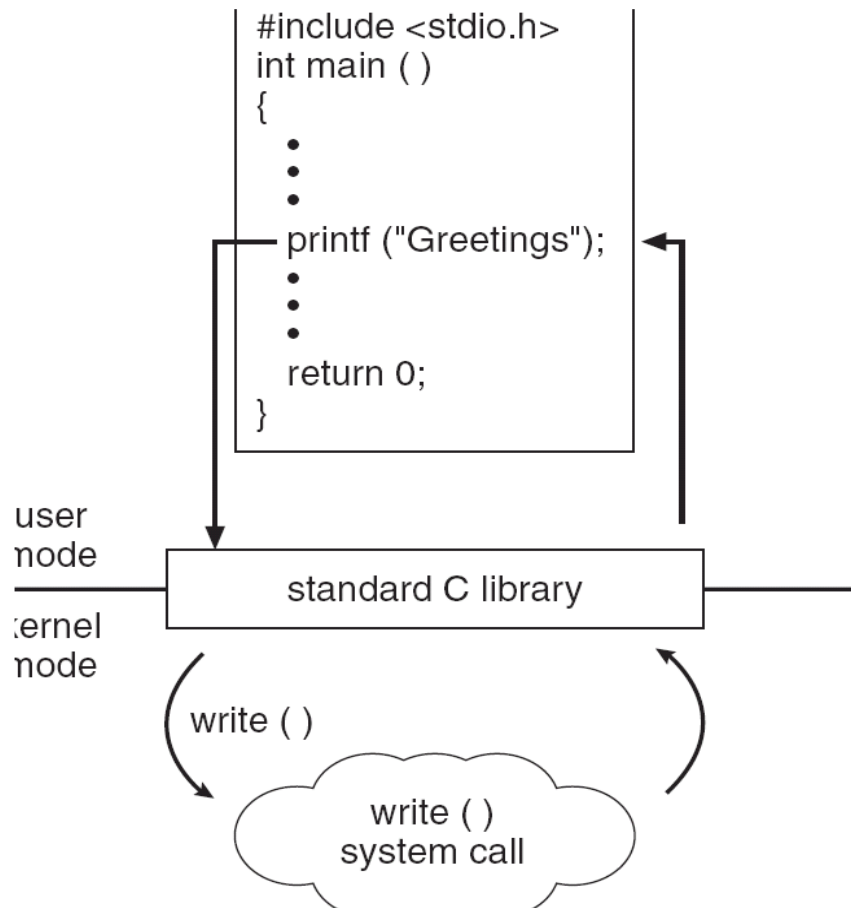
Application

System Calls: fork(), wait(), read(), open(), write(), mkdir(), kill() ...



Standard C Library Example

- C program invoking printf() library call, which calls write() system call



Resources Managed by OS

- Physical
 - CPU, Memory, Disk, I/O Devices like keyboard, monitor, printer
- Logical
 - Process, File, ...

Hence we have

1. Process management
2. Memory management
3. File management
4. I/O management